

University of Kelaniya

Faculty of Computing and Technology



Internet of Things - CTEC 32023

Mini-Project

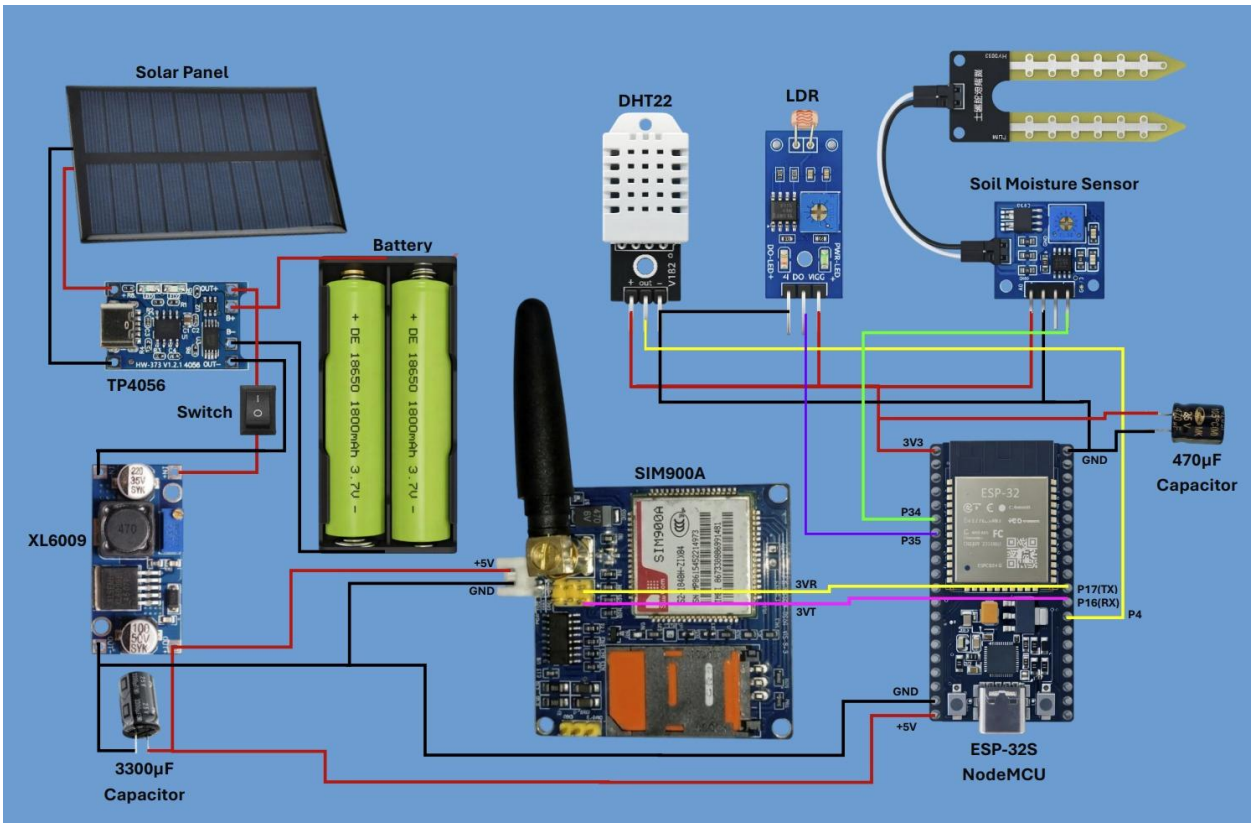
Off-Grid Smart Agriculture Node

Design and Implementation

Group Members:

- CT/2021/014 – W.A.K.H.Wijesingha
- CT/2021/015 – N.P. Wijewardhana
- CT/2021/045 – H.G.G.I.Ranaweera

01. Circuit Diagram and Hardware Setup



1. ESP32 Connections

Component	**ESP32Pin**
DHT22 Data	GPIO 4
Soil Moisture Sensor (A0)	GPIO 34
LDR Sensor	GPIO 35
SIM900A TX	GPIO 16 (RX2)
SIM900A RX	GPIO 17 (TX2)
VCC(Sensors)	3.3V / External Power
VCC(SIM900A)	5V
GND	GND

2. Power System

The off-grid power system consists of:

- Solar Panel (6V) → Connected to TP4056 Charging Module
- 18650 Battery (3.7V) → Connected to TP4056 Battery Terminals
- XL6009 Boost Converter → Steps up 3.7V to stable 5V output
- 5V Rail → Powers ESP32 and SIM900A
- 3.3V Rail → Powers sensors via ESP32 internal regulator

3. Hardware Setup

The sensors are connected to the ESP32 microcontroller. The ESP32 reads environmental data and sends it to the SIM800L GSM module. The GSM module uploads the data to the Thing Speak cloud platform using mobile network connectivity. The entire system is powered by a solar panel and rechargeable battery.

02. Source Code with Comments and Explanation

```
#define BLYNK_TEMPLATE_ID "TMPL6_cIRv-mg"
#define BLYNK_TEMPLATE_NAME "Off Grid Agry Node"
#define BLYNK_AUTH_TOKEN "rGYOZ-cxFLtzIlOXYxYEJZCDCZDb6xm4"
#define TINY_GSM_MODEM_SIM800 // SIM800 library works perfectly for SIM900A
#include <TinyGsmClient.h>
#include <BlynkSimpleTinyGSM.h>
#include <DHT.h>

// Network Settings

const char apn[]      = "hutch3g";
const char gprsUser[] = "";

const char gprsPass[] = "";

// Hardware Pins

#define SerialAT Serial2
#define SIM_RX   16

#define SIM_TX   17
#define SIM_BAUD 9600
#define DHTPIN       4
#define DHTTYPE      DHT22
#define SOIL_PIN     34
#define LDR_PIN      35

// Deep Sleep Settings

#define TIME_TO_SLEEP  300     // 300 seconds = 5 minutes
#define uS_TO_S_FACTOR 1000000ULL    // Conversion factor for microseconds

DHT dht(DHTPIN, DHTTYPE);

 TinyGsm modem(SerialAT);

// Function to keep sensor percentages between 0 and 100

int mapConstrained(int value, int fromLow, int fromHigh, int toLow, int toHigh) {
int result = map(value, fromLow, fromHigh, toLow, toHigh);
return constrain(result, toLow, toHigh); }
```

```
void setup() {

// 1. STARTUP DELAY

// Give the hardware a moment to stabilize after waking up from sleep
delay(2000);

dht.begin();

// 2. READ SENSORS FIRST

// We read data before turning on the internet to save power
float humidity = dht.readHumidity();
float temperature = dht.readTemperature();
int soilRaw = analogRead(SOIL_PIN);
int ldrRaw = analogRead(LDR_PIN);
int soilPercent = mapConstrained(soilRaw, 3800, 800, 0, 100);
int lightPercent = mapConstrained(ldrRaw, 4095, 100, 0, 100);

// If the DHT22 fails to read, go immediately back to sleep to save battery
if (isnan(humidity) || isnan(temperature)) {
esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
esp_deep_sleep_start(); }

// 3. WAKE UP THE MODEM

SerialAT.begin(SIM_BAUD, SERIAL_8N1, SIM_RX, SIM_TX);
delay(3000);
TinyGsmAutoBaud(SerialAT, 9600, 9600);

modem.init();

// 4. CONNECT TO HUTCH & BLYNK

modem.waitForNetwork(60000L);
modem.gprsConnect(apn, gprsUser, gprsPass);
// Use Port 8080 to ensure a stable connection over 2G
Blynk.config(modem, BLYNK_AUTH_TOKEN, "blynk.cloud", 8080);

Blynk.connect();

// 5. SEND DATA
```

```
if (Blynk.connected()) {
Blynk.virtualWrite(V0, temperature);

Blynk.virtualWrite(V1, humidity);

Blynk.virtualWrite(V2, soilPercent);
Blynk.virtualWrite(V3, lightPercent);
// Process the data transfer
Blynk.run();
delay(1000); }

// 6. SHUT DOWN NETWORK

// Disconnect the GPRS to stop the modem from draining heavy power
modem.gprsDisconnect();



// 7. ENTER DEEP SLEEP

// Tell the ESP32 timer to wake it up in 5 minutes
esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
esp_deep_sleep_start(); }
void loop() {
// The loop remains completely empty.

// When the ESP32 wakes up from deep sleep, it automatically starts over at the top of setup()!

}
```

03. Integration Process

Step 1: Power System Setup

- Connect solar panel to TP4056 charging module.
- Connect rechargeable battery.
- Verify charging operation.

Step 2: Sensor Integration

- Connect DHT11 sensor.
- Connect Soil Moisture Sensor.
- Connect LDR Sensor.
- Test all sensors individually.

Step 3: ESP32 Programming

- Install Arduino IDE.
- Install required libraries.
- Upload test code.
- Verify sensor readings through Serial Monitor.

Step 4: GSM Communication Setup

- Connect SIM800L module.
- Insert activated SIM card.
- Configure APN settings.
- Test internet connection.

Step 5: Cloud Integration

- Create Thing Speak account.
- Create data channel.
- Obtain API key.
- Upload sensor data to cloud.

Step 6: Dashboard Monitoring

- View data graphs.
- Monitor environmental conditions remotely.

04. Testing and Debugging Process

Sensor Testing

- Compare sensor readings with actual environmental conditions.
- Check accuracy of temperature and humidity values.

GSM Module Testing

- Verify network registration.
- Test data transmission to cloud server.

Power Testing

- Check battery charging process.
- Measure battery voltage.
- Confirm solar charging during daytime.

System Testing

- Run the system continuously for 24 hours.
- Verify successful data uploads.
- Monitor battery performance.

Debugging Challenges and Solutions

Challenge	Solution
2A spikes caused modem reboots.	3300µF capacitor across 5V/GND.
Noise caused erratic analog readings.	470µF capacitor on 3.3V rail.
No common reference → corrupted UART data.	Tied all GNDs together.
Continuous operation drained batteries.	Deep sleep, wake every 5 minutes.
Linear charging caused overheating.	Aluminum heatsink on the IC.